第三章

栈和队列

主要内容

3.1 栈的特性和基本操作

3.2 栈的应用举例

3.3 队列的特性和基本操作

3.4 队列的应用举例

通常称栈和队列是限定插入和删除只能在表的“端点”进行的线性表。

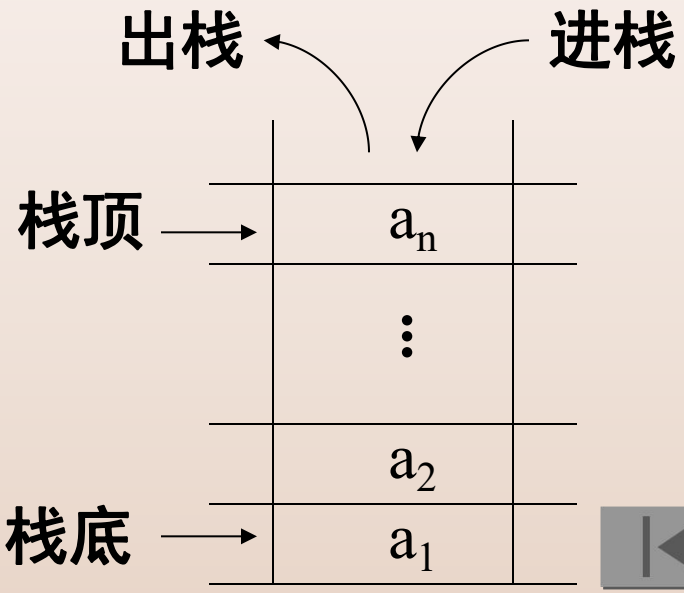
线性表	栈	队列
Insert(L, **i**, x)	Insert(S, **n+1**, x)	Insert(Q, **n+1**, x)
$1 \le i \le n+1$		
Delete(L, **i**)	Delete(S, **n**)	Delete(Q, **1**)
$1 \le i \le n$		

栈和队列是两种常用的数据类型

3.1 栈的特性和基本操作

一、堆栈的定义

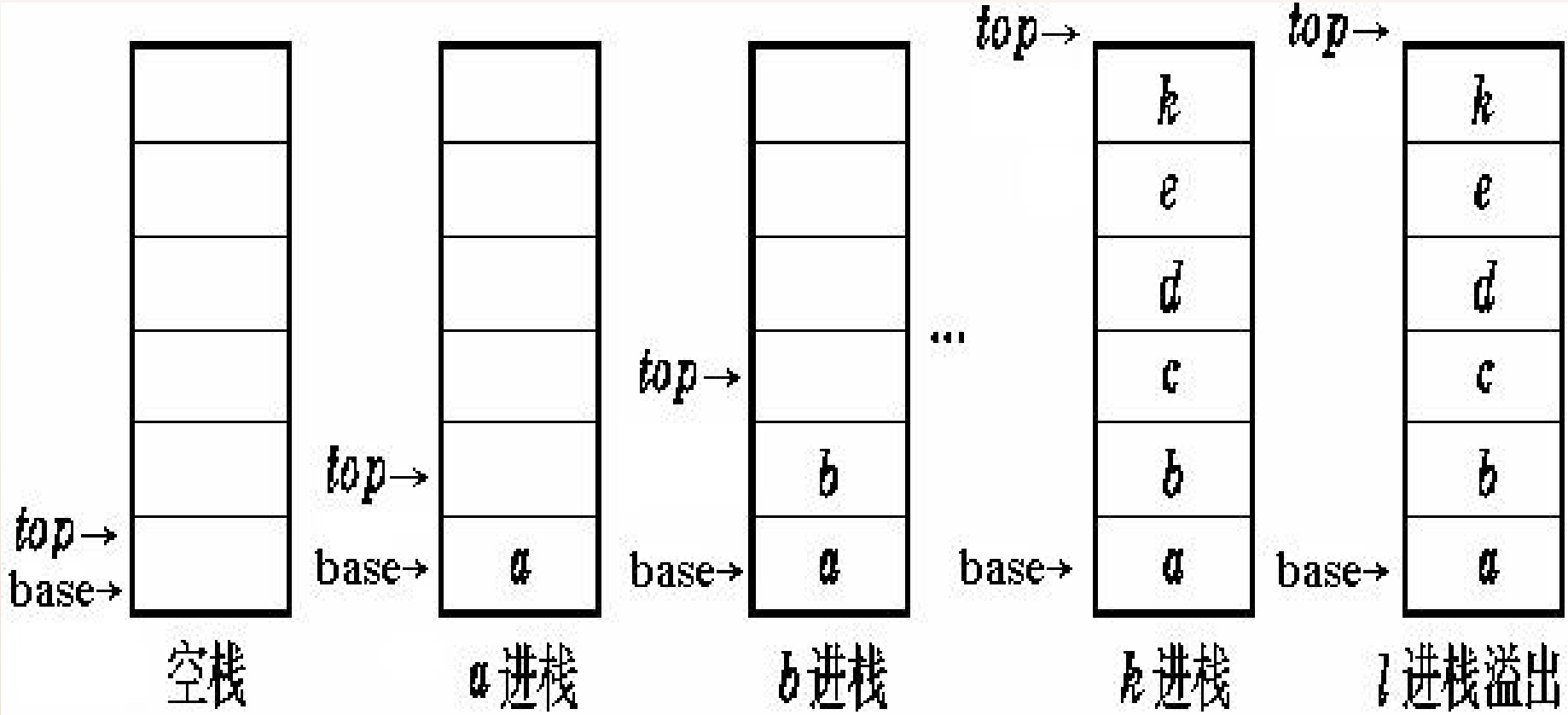
- **栈：** 是限定在**表尾**进行插入或删除操作的线性表。
- **栈顶：** 表尾端，元素从这儿插入和删除。
- **栈底：** 表头端。
- 栈的修改是按**后进先出**（Last In First Out）的原则进行的。



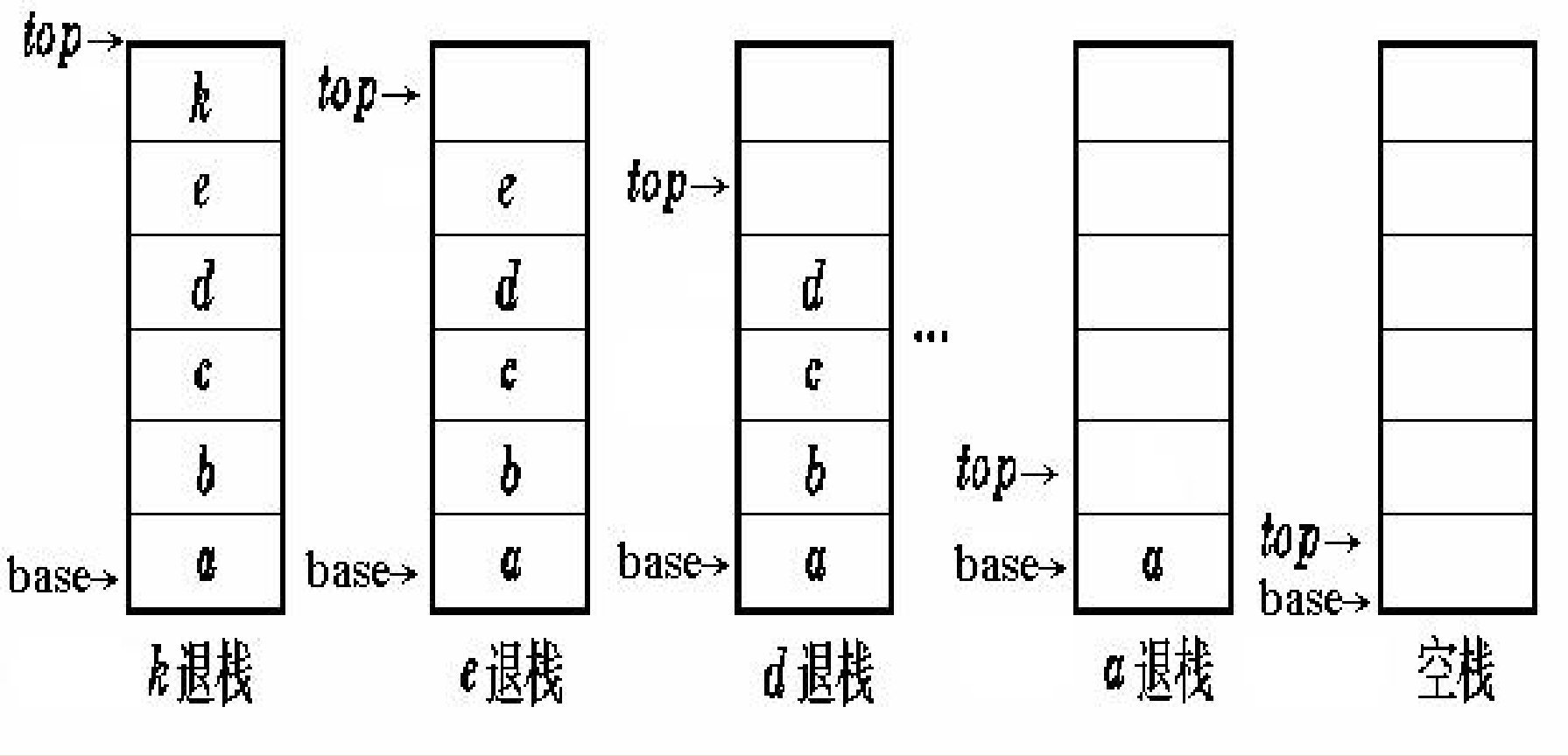
二、堆栈的特点

- 根据堆栈的定义，每次删除的总是堆栈中当前的栈顶元素，即最后进入堆栈的元素。
- 在进栈时，最先进入的元素一定在栈底，最后进入的元素一定在栈顶。
- 由于这一特点，称堆栈是后进先出表或下推表（LIFO-Last In First Out）

进栈示例


top→
base→
空栈
top→
base→
a
a进栈
top→
b
a
base→
b进栈
...
top→
k
e
d
c
b
a
base→
k进栈
top→
k
e
d
c
b
a
base→
l进栈溢出

出栈示例


top→
h
e
d
c
b
base→
a
h退栈
top→
e
d
c
b
base→
a
e退栈
top→
d
c
b
base→
a
d退栈
…
top→
base→
a
a退栈
top→
base→
空栈

栈的抽象数据类型定义：

ADT Stack {

数据对象：

$D = \{ a_i \mid a_i \in ElemSet, i=1,2,\ldots,n, \ n \geq 0 \}$

数据关系：

$R1 = \{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D, i=2,\ldots,n \}$

约定a_n端为栈顶，a_1端为栈底。

基本操作：

} ADT Stack

- **InitStack(&S)**

  操作结果：构造一个空栈 S。

- **DestroyStack(&S)**

  初始条件：栈 S 已存在。

  操作结果：栈 S 被销毁。

- **ClearStack(&S)**

  初始条件：栈 S 已存在。

  操作结果：将 S 清为空栈。

- **StackLength(S)**

  初始条件：栈 S 已存在。

  操作结果：返回 S 的元素个数，即栈的长度。

- **StackEmpty(S)**

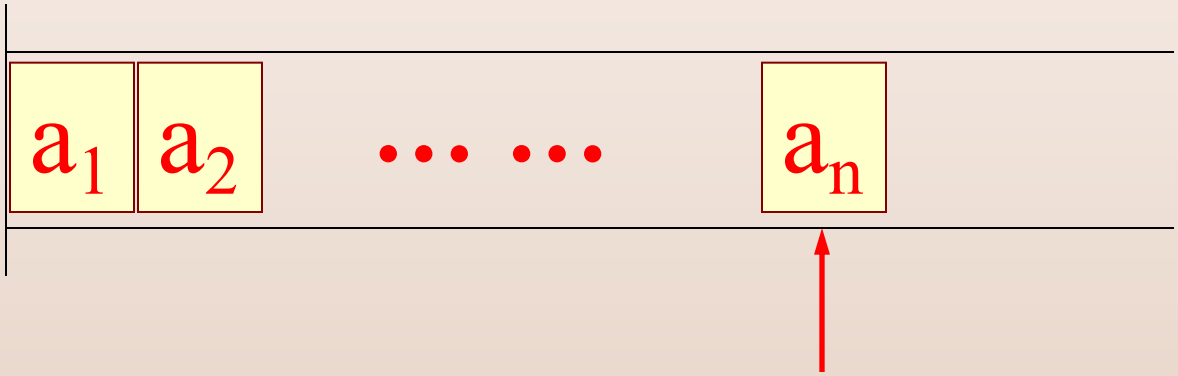
  初始条件：栈 S 已存在。

  操作结果：若栈 S 为空栈，则返回 TRUE，否则 FALE。

- **GetTop(S, &e)**

  初始条件：栈 S 已存在且非空。

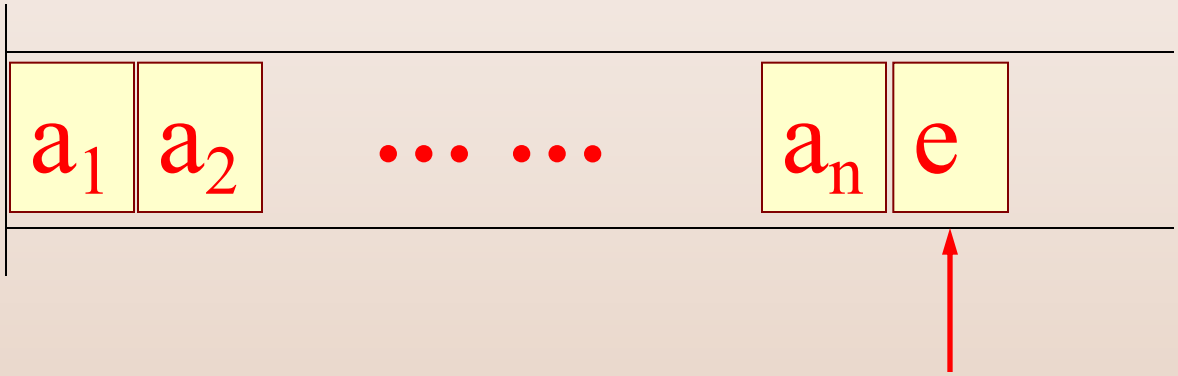
  操作结果：用 e 返回 S 的栈顶元素。



- **Push(&S, e)**

初始条件：栈 S 已存在。

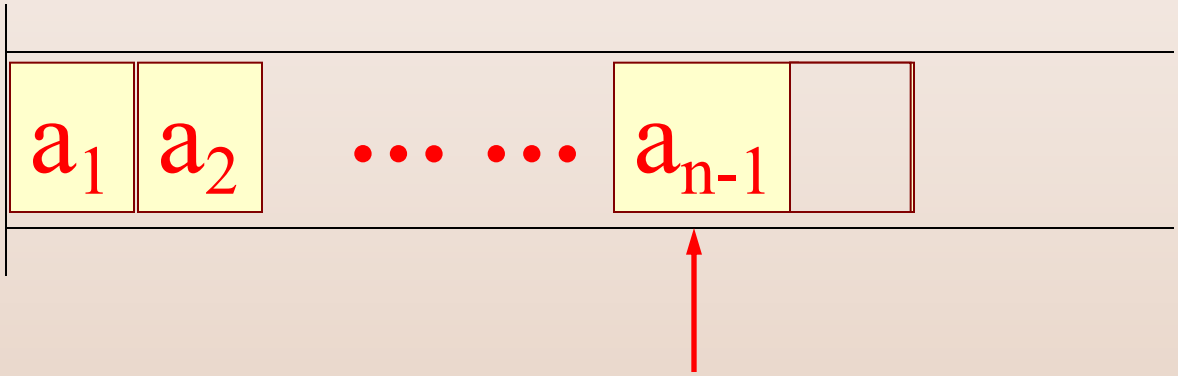
操作结果：插入元素 e 为新的栈顶元素。



- **Pop(&S, &e)**

  初始条件：栈 S 已存在且非空。

  操作结果：删除 S 的栈顶元素，并用 e 返回其值。



- StackTraverse(S,visit( ))

  初始条件：栈 S 已存在且非空。

  操作结果：从栈底到栈顶依次对S的每个数据元素调用函数visit。一旦visit调用失败，则操作失败。

栈类型的实现

顺序栈

链栈

顺序栈的定义

- 借助一维数组来存储栈中元素的堆栈一般称为顺序栈。
- 设一维数组为STACK[M]
- 再设一个整型变量top表示栈顶指针。

  以 top＝0 表示空栈。

  非空栈中的top指针始终指向栈顶元素的下一个位置。
- STACK[0]为第一个进入堆栈的元素， 当没有删除运算时，STACK[i-1]为第i个进入堆栈的元素，STACK[top-1]为栈顶元素。

顺序栈的定义

- 由于堆栈是一个动态结构，因此有溢出问题，当堆栈中已经有M个元素时，如果再做进栈运算则会产生溢出，称之为上溢；对空栈进行删除运算也会产生溢出，称之为下溢。
- 为了避免溢出，在对堆栈进行进栈运算和退栈运算之前都应该分别测试堆栈是否已满或者是否已空。

类似于线性表的顺序映象实现，指向表尾(栈顶)的指针可以作为栈顶指针

```
//----- 栈的顺序存储表示 -----
 #define  STACK_INIT_SIZE  100;
 #define  STACKINCREMENT   10;
 typedef struct {
   SElemType  *base;  //栈底指针
   SElemType  *top;   //栈顶指针
   int  stacksize;   //当前已分配的存储空间
 } SqStack;
```

```
Status InitStack (SqStack &S)
{ // 构造一个空栈S
  S.base=(ElemType*)malloc(STACK_INIT_SIZE*
                                    sizeof(ElemType));
  if (!S.base) exit (OVERFLOW); //存储分配失败
  S.top = S.base;
  S.stacksize = STACK_INIT_SIZE;
  return OK;
}
```

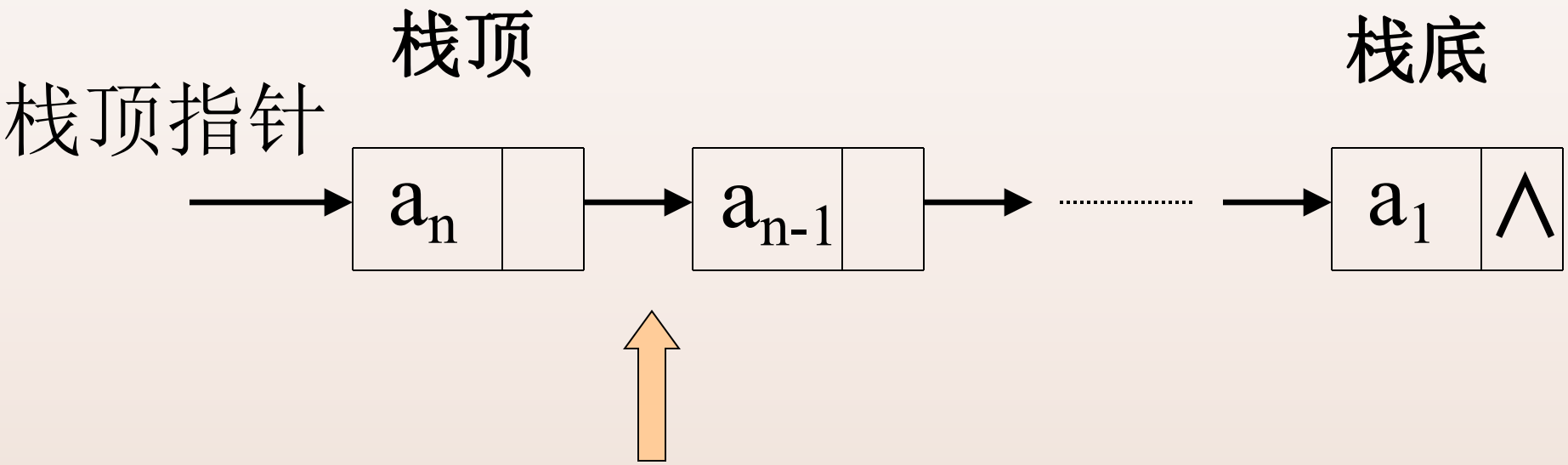
```
Status Push (SqStack &S, SElemType e) {
  if (S.top - S.base >= S.stacksize) {//栈满，追加存储空间
    S.base = (ElemType *) realloc ( S.base,
            (S.stacksize + STACKINCREMENT) *
                                    sizeof (ElemType));
    if (!S.base) exit (OVERFLOW); //存储分配失败
    S.top = S.base + S.stacksize;
    S.stacksize += STACKINCREMENT;
  }
  *S.top++ = e;
  return OK;
}
```

```
Status Pop (SqStack &S, SElemType &e) {
    // 若栈不空，则删除S的栈顶元素，
    // 用e返回其值，并返回OK;
    // 否则返回ERROR
   if (S.top == S.base) return ERROR;
   e = *--S.top;
   return OK;
}
```

链 栈

- 链式栈就是用一个线性链表来实现一个堆栈结构。栈中每个元素用一个链结点表示，同时，设置一个指针变量，指出当前栈顶元素所在结点的存储位置，当栈为空时，有top==NULL，下图就是链接栈的一般形式：

链栈示意图



注意：链栈中指针的方向

链栈的特点

- 链表的头结点就是栈顶元素的结点
- 根据堆栈的定义，新结点的插入和栈顶结点的删除都在表头进行
  - 插入一个新元素，相当于在第一个结点之前插入一个新结点
  - 删除链接栈的栈顶元素，就是删除链表的第一个结点。
- 不会有栈满而产生溢出的问题

3.2 栈的应用举例

一、表达式求值

把一个表达式翻译成正确求值的一个机器指令序列，或者直接对表达式求值，首先应正确解释表达式。

（1） 假定所有运算分量都是整数；

（2） 所有运算符都是整数的二元操作，且都用一个字符表示。

算术四则运算规则：

(1)先乘除后加减；

(2)左结合性；

(3)先括号内后括号外。

运算符优先关系表

- + − * /为θ_1时，优先性均低于“(”，高于“)”
- $\theta_1 = \theta_2$时，令$\theta_1 > \theta_2$;
- #是表达式的开始和结束符。

θ_1 \ θ_2	+	-	*	/	(	)	#
+	>	>	<	<	<	>	>
-	>	>	<	<	<	>	>
*	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	
)	>	>	>	>		>	>
#	<	<	<	<	<		=

算法的基本思想：

- 使用两个栈，一个寄存运算符——OPTR，另一个寄存操作数或运算结果——OPND。
- 基本思想是：

1 首先将OPND置为空栈，将表达式起始符#作为OPTR的栈底元素；

2 顺序扫描表达式，当读入一个运算分量时就将其放进OPND，而在读入一个运算符时，则和OPTR栈的栈顶运算符比较优先权后作相应操作，直到整个表达式求值完毕（即OPTR栈顶元素和当前读入的字符均为# ）。

表达式求值

从左向右扫描表达式：

遇操作数——保存；

遇运算符号 θ_2——与前面的运算符 θ_1 比较

若 $\theta_1 < \theta_2$ 则保存 $\theta 2$；

若 $\theta_1 > \theta_2$ 则 $\theta 1$ 可进行运算；

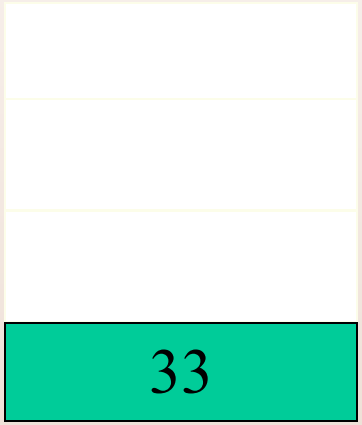
若 $\theta_1 = \theta_2$ 需要消去括号；

```
OperandType EvaluateExpression( ) {
//算术表达式求值的运算符优先算法。OPTR和OPND
  分别为运算符栈和运算数栈，OP为运算符集合
InitStack(OPTR);    Push(OPTR,'#');
InitStack(OPND);    c=getchar( );
While(c!='#'||GetTop(OPTR)!='#')  {
   if(!In(c,OP)) {Push((OPND,c); c=getchar( ); }
   else
      switch( Precede( GetTop( OPTR ), c )) {
// 比较OPTR栈顶运算符与新读入的运算符的优先级
         case '<':      // 栈顶元素优先权低
               Push(OPTR,c); c=getchar( ); break;
```

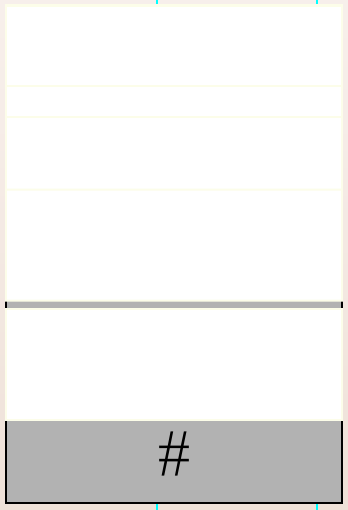
```
        case '=':      // 脱下括号并接收下一个字符
            Pop(OPTR,x); c=getchar( ); break;
        case '>':      // 退栈并将运算结果入栈
            Pop(OPTR,theta);   Pop(OPND,b);
            Pop(OPND,a);
            Push(OPND,Operate(a,theta,b));  break;
        }  // switch
    }  // while
    return GetTop(OPND);
}  // EvaluateExpression
```

例：

输入：3 * （ 7 + 3 * 6 / 2 - 5 ） #



OPND



OPTR

§3.3 递归实现

当在一个函数的运行期间调用另一个函数时，在运行该被调用函数之前，需先完成三项任务：

- 将所有的实在参数、返回地址等信息传递给被调用函数保存；
- 为被调用函数的局部变量分配存储区；
- 将控制转移到被调用函数的入口。

从被调用函数返回调用函数之前，应该完成下列三项任务：

- 保存被调函数的计算结果；
- 释放被调函数的数据区；
- 依照被调函数保存的返回地址将控制转移到调用函数。

多个函数嵌套调用的规则是：

后调用先返回！

此时的内存管理实行“栈式管理”

例如：

```
void main( ){    void a( ){      void b( ){
   …                …               …
   a( );            b( );
   …                …
}//main          }// a          }// b
```

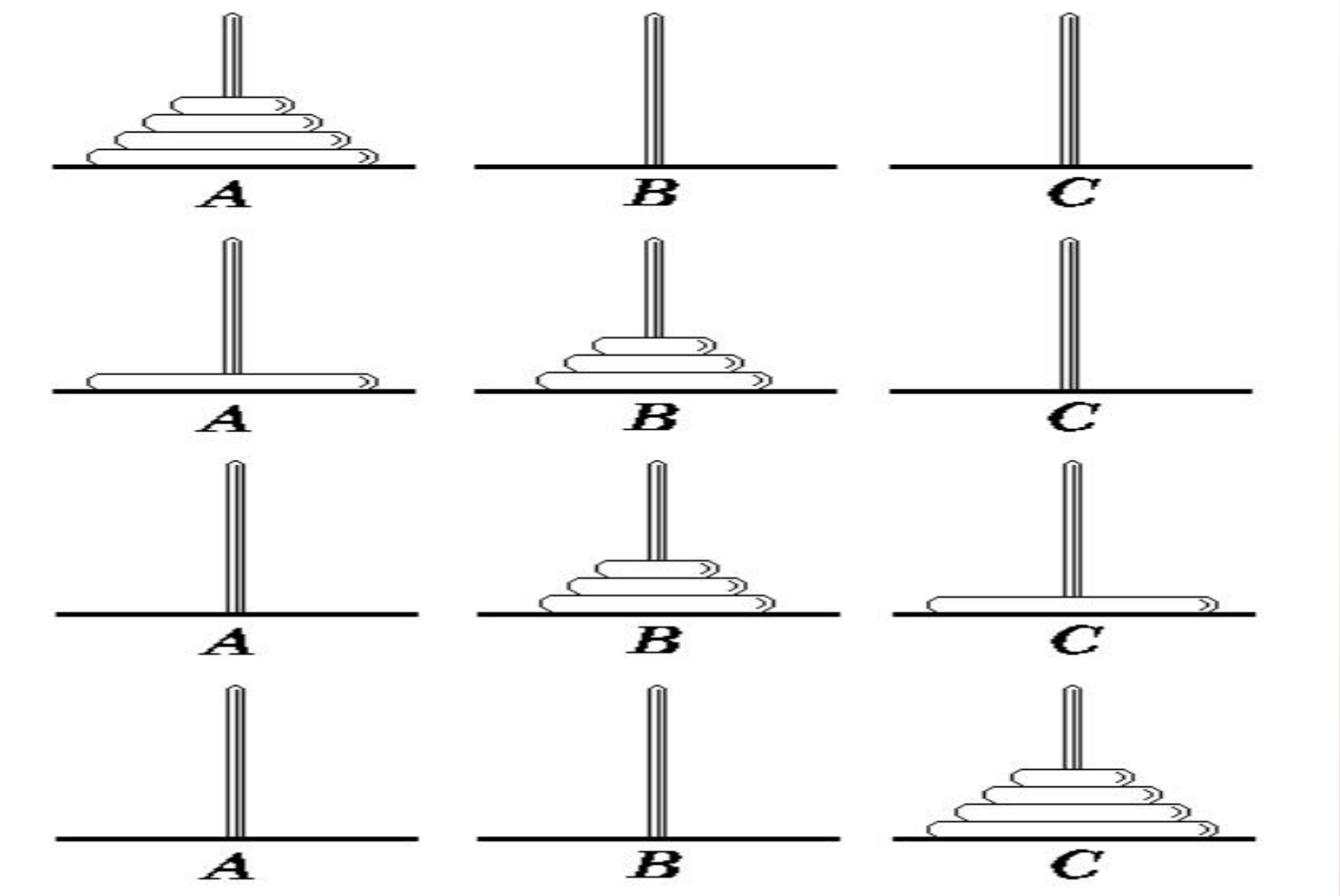
函数a的数据区

Main的数据区

递归函数执行的过程可视为同一函数进行嵌套调用

例如：

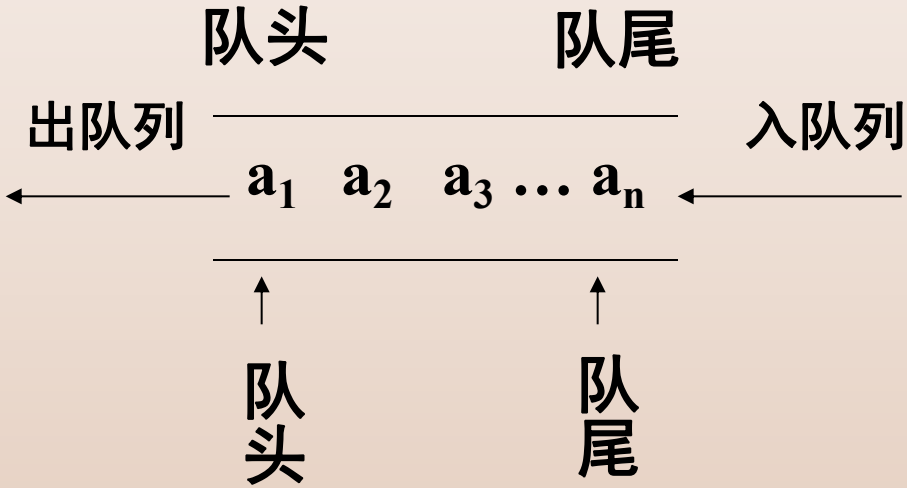
例如，汉诺塔(Tower of Hanoi)问题



```
void hanoi (int n, char x, char y, char z) {
// 将塔座x上按直径由小到大且至上而下编号为1至n
// 的n个圆盘按规则搬到塔座z上，y可用作辅助塔座。
1  if (n==1)
2  move(x, 1, z);     // 将编号为 1 的圆盘从x移到z
3  else {
4      hanoi(n-1, x, z, y);     // 将x上编号为 1 至n-1的
                                //圆盘移到y, z作辅助塔
5      move(x, n, z);           // 将编号为n的圆盘从x移到z
6      hanoi(n-1, y, x, z);     // 将y上编号为 1 至n-1的
                                //圆盘移到z, x作辅助塔
7   }
8  }
```

§3.4 队列

队列也是一种特殊的线性表，对于它所有的插入都在表的一端进行，所有的删除都在表的另一端进行。进行删除的这一端叫队列的头，进行插入的这一端叫队列的尾。当队列中没有任何元素时，称为空队列。队列也称作先进先出表。



- 假设Q=(a_1, a_2, …, a_{n-1}, a_n)为一个队结构，则队头为a_1，队尾为a_n。该队列是按a_1, a_2, …, a_{n-1}, a_n的顺序进入的，退出该队列也只能按这个顺序进行。
- 队列和堆栈一样，也是动态结构，同样存在溢出问题，在进行插入、删除运算之前，应进行队满、队空的判断。

队列的抽象数据类型定义：

ADT Queue {

数据对象：

$$D = \{a_i \mid a_i \in ElemSet,\ i=1,2,\ldots,n,\ n \geq 0\}$$

数据关系：

$$R1 = \{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D,\ i=2,\ldots,n\}$$

约定其中a_1端为队列头， a_n端为队列尾

基本操作：

} **ADT** Queue

队列的基本操作：

InitQueue(&Q)　　DestroyQueue(&Q)

QueueEmpty(Q)　　QueueLength(Q)

GetHead(Q, &e)　　ClearQueue(&Q)

EnQueue(&Q, e)　　DeQueue(&Q, &e)

InitQueue(&Q)

操作结果：构造一个空队列Q。

DestroyQueue(&Q)

初始条件：队列Q已存在。

操作结果：队列Q被销毁，不再存在。

QueueEmpty(Q)

初始条件：队列Q已存在。

操作结果：若Q为空队列，则返回TRUE，否则返回FALSE。

QueueLength(Q)

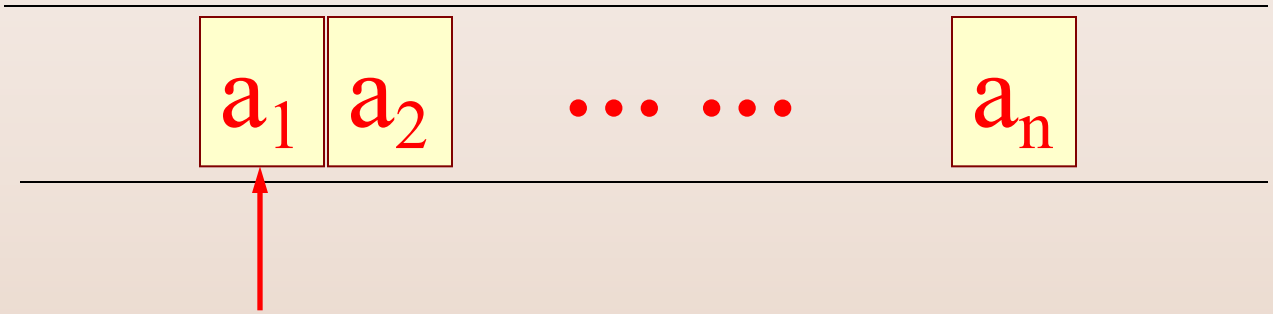
初始条件：队列Q已存在。

操作结果：返回Q的元素个数，即队列的长度。

GetHead(Q, &e)

初始条件： Q为非空队列。

操作结果： 用e返回Q的队头元素。



ClearQueue(&Q)

初始条件：队列Q已存在。

操作结果：将Q清为空队列。

QueueTravers(Q, visit())

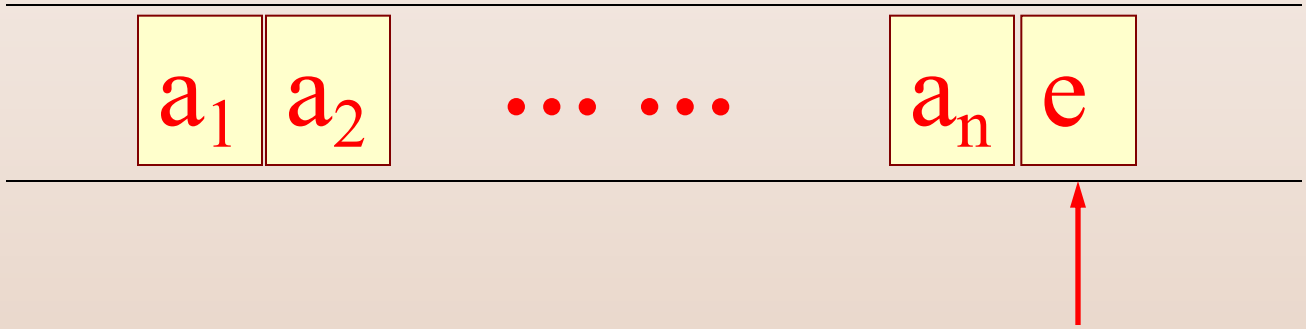
初始条件：队列Q已存在且非空。

操作结果：从队头到队尾，依次对Q的每个数据元素调用函数visit，一旦visit调用失败，则操作失败。

EnQueue(&Q, e)

初始条件： 队列Q已存在。

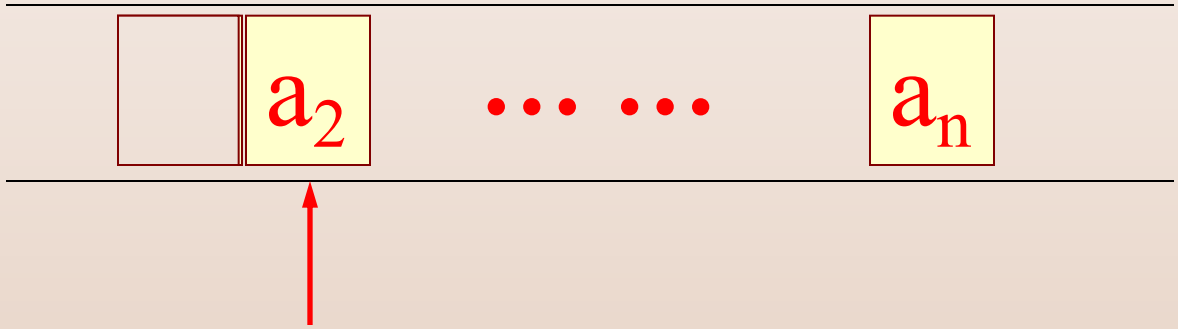
操作结果： 插入元素e为Q的新的队尾元素。



DeQueue(&Q, &e)

初始条件： Q为非空队列。

操作结果： 删除Q的队头元素，并用e返回其值。



队列类型的实现

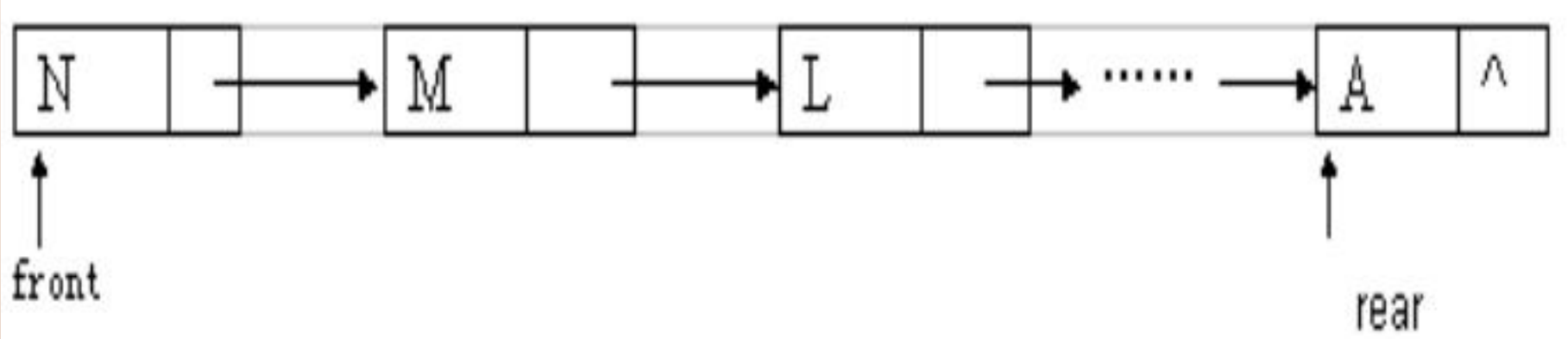
链队列——链式映象

循环队列——顺序映象

链式队列

- 队列的链式存储结构就是用一个线性链表来表示队列，简称链式队。
- 把线性链表的头指针定义为队头指针front，把链表的最后结点指针rear作为队尾指针，并且限定只能在链头进行删除，只能在链尾进行插入。这个线性链表就构成了一个链接队列。

- 下图为链接队列的一般形式：
- 测试链式存储结构队列是否为空的条件为 front ==NULL
- 插入一个新元素就是在表尾结点后添加一个新结点
- 删除一个元素就是删除链表的第一个结点

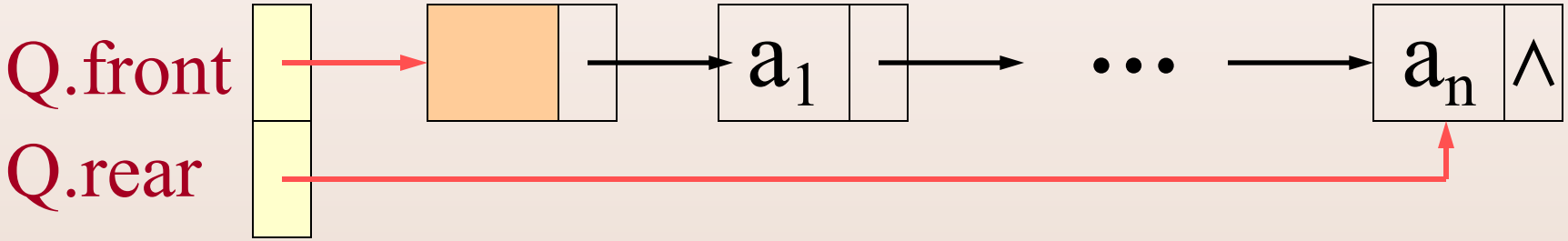


- 队头在链头，队尾在链尾。
- 链式队列在进队时无队满问题，但有队空问题。
- 队空条件为 *front == NULL*

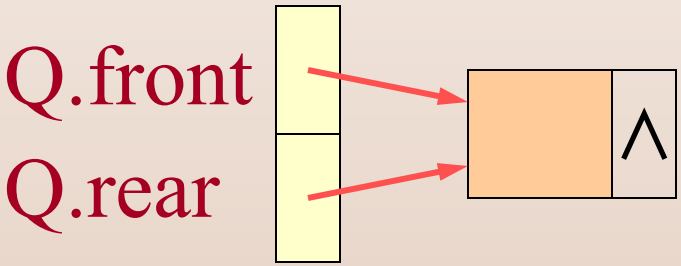
链队列——链式映象

```
typedef struct QNode {// 结点类型
  QElemType      data;
  struct QNode  *next;
} QNode, *QueuePtr;
```

```
typedef struct { // 链队列类型
    QueuePtr  front;  // 队头指针
    QueuePtr  rear;   // 队尾指针
} LinkQueue;
```



空队列



```
Status InitQueue (LinkQueue &Q) {
  // 构造一个空队列Q
  Q.front = Q.rear =
            (QueuePtr)malloc(sizeof(QNode));
  if (!Q.front) exit (OVERFLOW);
                                  //存储分配失败
  Q.front->next = NULL;
  return OK;
}
```

```
Status EnQueue (LinkQueue &Q,
                                QElemType e) {
  // 插入元素e为Q的新的队尾元素
  p = (QueuePtr) malloc (sizeof (QNode));
  if (!p)  exit (OVERFLOW);
                                //存储分配失败
  p->data = e;   p->next = NULL;
  Q.rear->next = p;    Q.rear = p;
  return OK;       }
```

```
Status DeQueue (LinkQueue &Q,
                                QElemType &e) {
 // 若队列不空，则删除Q的队头元素，
 //用 e 返回其值，并返回OK；否则返回ERROR
  if (Q.front == Q.rear)    return ERROR;
  p = Q.front->next;   e = p->data;
  Q.front->next = p->next;
  if (Q.rear == p)  Q.rear = Q.front;
  free (p);       return OK;
}
```

队列的顺序表示和实现—循环队列

- 可以用一维数组来描述队列的顺序存储结构。
- 首先定义一维数组elements[maxsize]来存放队列元素，同时还需要设置两个指针front与 rear分别指示队头元素和队尾元素的位置。

约定：初始化建空队列时，令

front=rear= 0

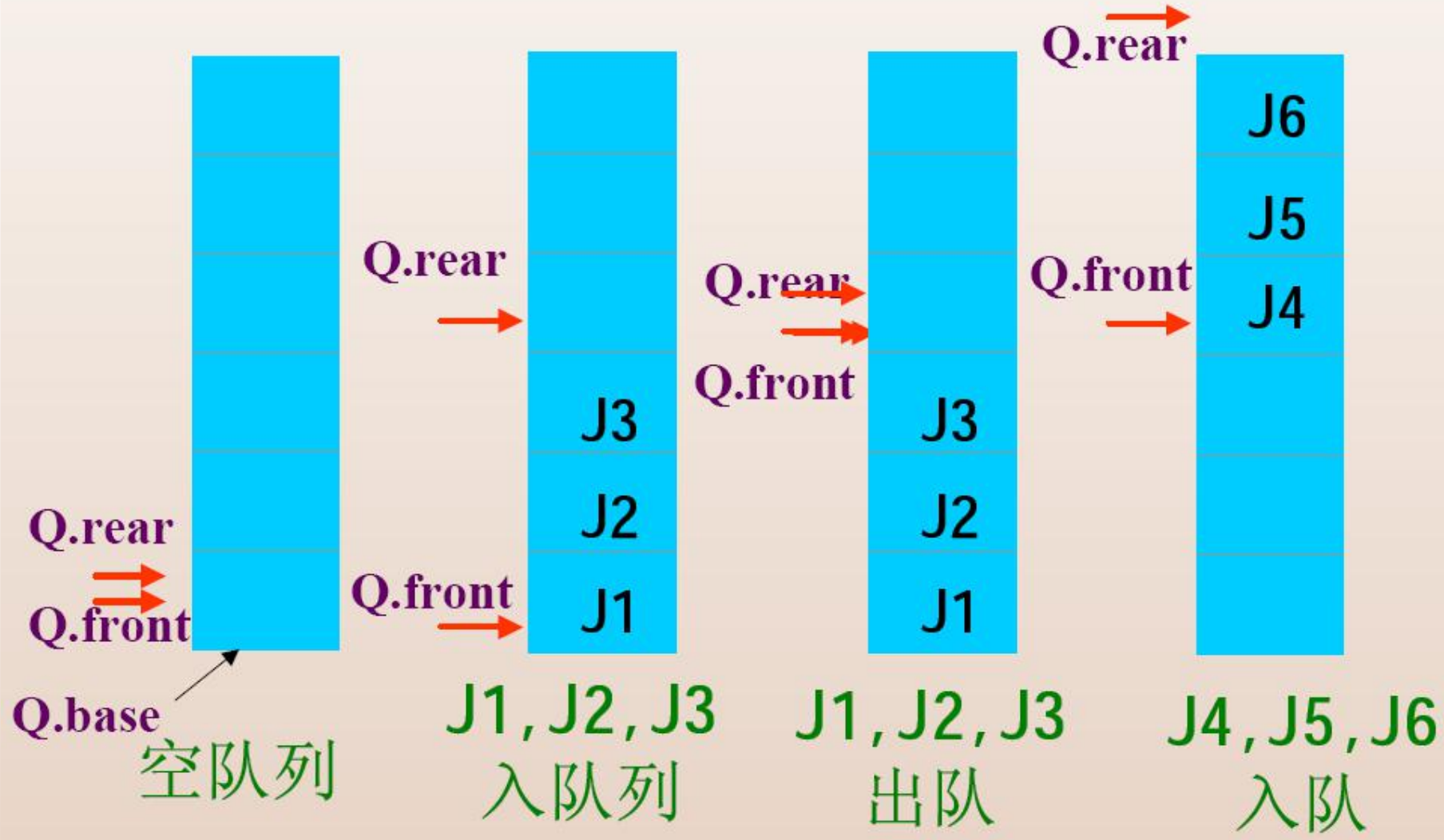
每当插入新的队列尾元素时“尾指针加1”，每当删除队列头元素时“头指针加1”。

在非空队列中，头指针始终指向队列头元素，尾指针始终指向队列尾元素的下一个位置。

测试一个队列是否为空的条件是

rear= =front

队列的示意图


Q.rear
Q.front
Q.base
空队列
Q.rear
Q.front
J3
J2
J1
J1, J2, J3
入队列
Q.rear
Q.front
J3
J2
J1
J1, J2, J3
出队
Q.rear
Q.front
J6
J5
J4
J4, J5, J6
入队

- 进行插入运算，必须先测试队满与否。若队满，则调用相关算法处理有关溢出问题；否则，将队尾指针加1，然后将新元素插入到当前队尾指针所指的位置。
- 删除队头元素，必须先测试队列是否为空。若队空，则调用相关算法处理有关信息；否则，删除队头元素(队头指针加1)，如果需要，可以把被删除元素保存起来。

- 在队列的插入算法中，当队尾指针rear==maxsize时，再做插入运算好象会产生溢出，而实际上这时队列的前端还有许多空的位置，因此，这种溢出称为假溢出。
- 解决假溢出问题可以采用循环队列的方法

可以把队列设想成头尾相连的循环表，即把存储队列元素的表从逻辑上看成一个环，这种队列通常称为循环队列。

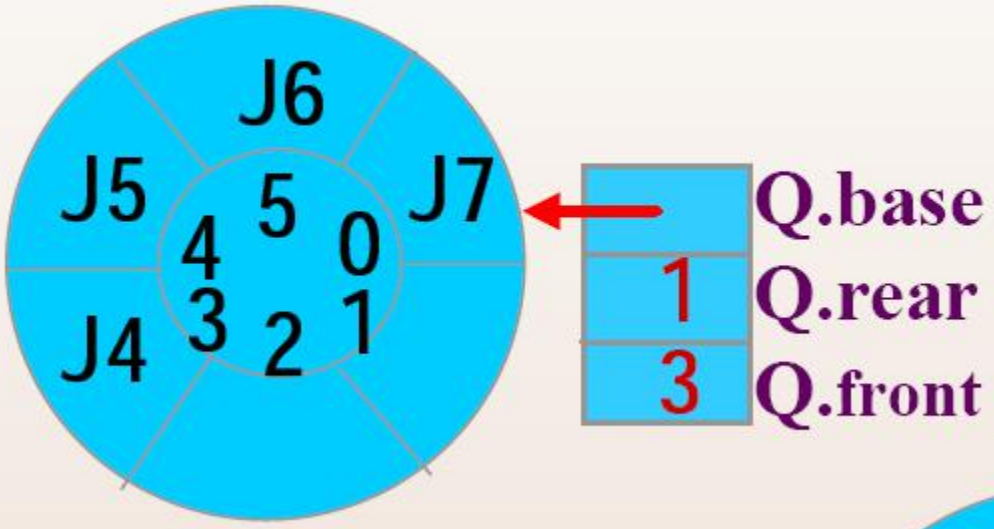
- 循环队列的首尾相接，当队头指针front和队尾指针rear进到maxsize-1时，再前进一个位置就自动到0，这可以利用除法取余的运算来实现。
- 插入中修改队尾指针的语句可以写成：

rear=(rear+1)% maxsize

- 删除中修改队头指针的语句可以写成：

front=(front+1)% maxsize

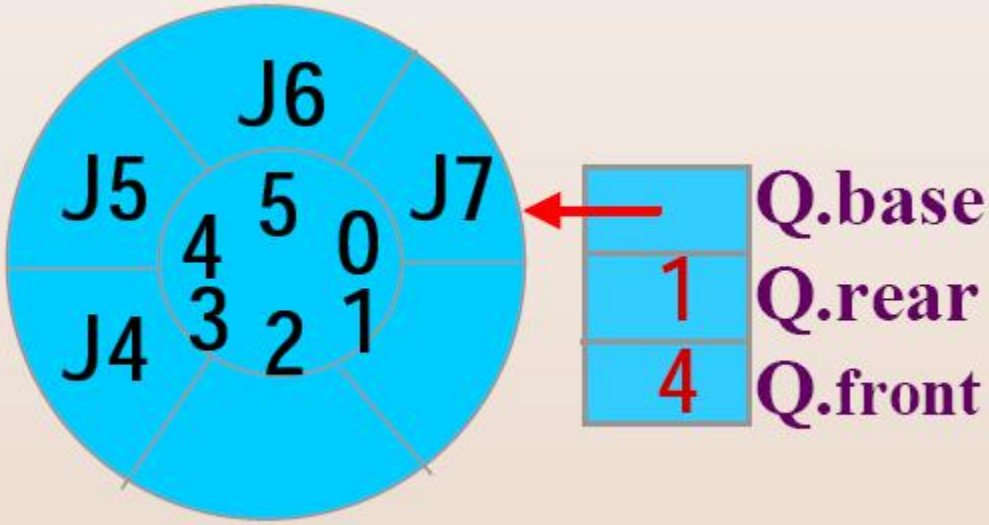
Q.rear=(Q.rear+1)%MAXQSIZE



出队操作前

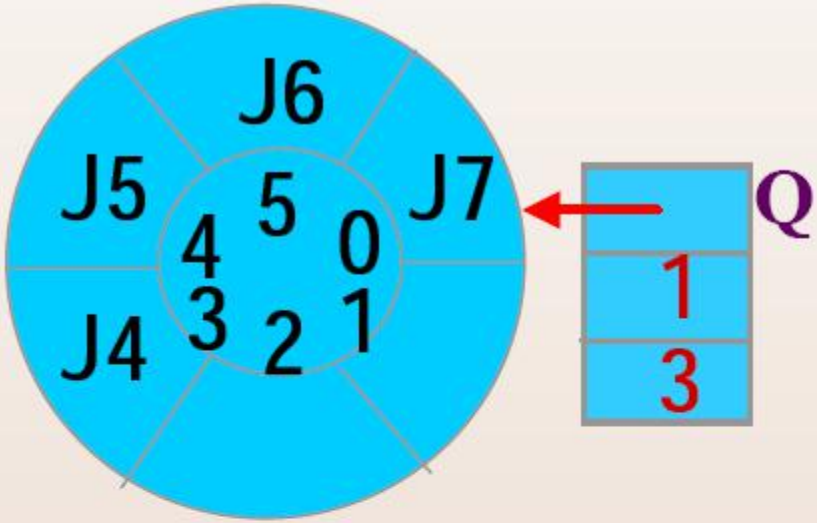
e J4

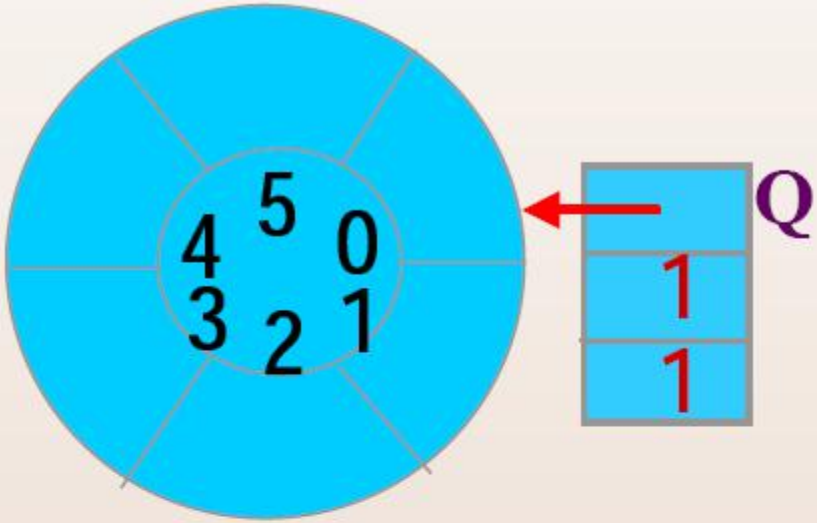
出队操作后

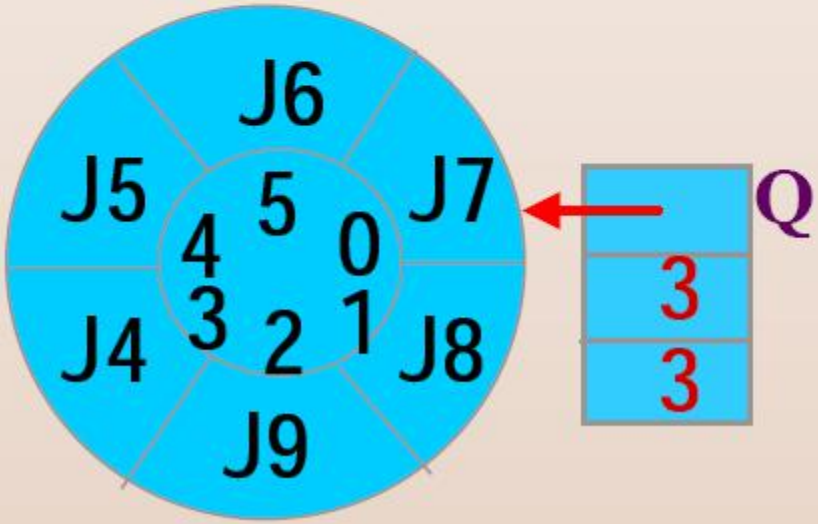


Q.front=(Q.front+1)% MAXQSIZE

循环队列判空、判满的方法及条件







Q.rear = =Q.front

方法一

另设标志

设置标志位判断队列空、满方法：

标识是由于rear增1使

rear＝front，还是由于front增1使

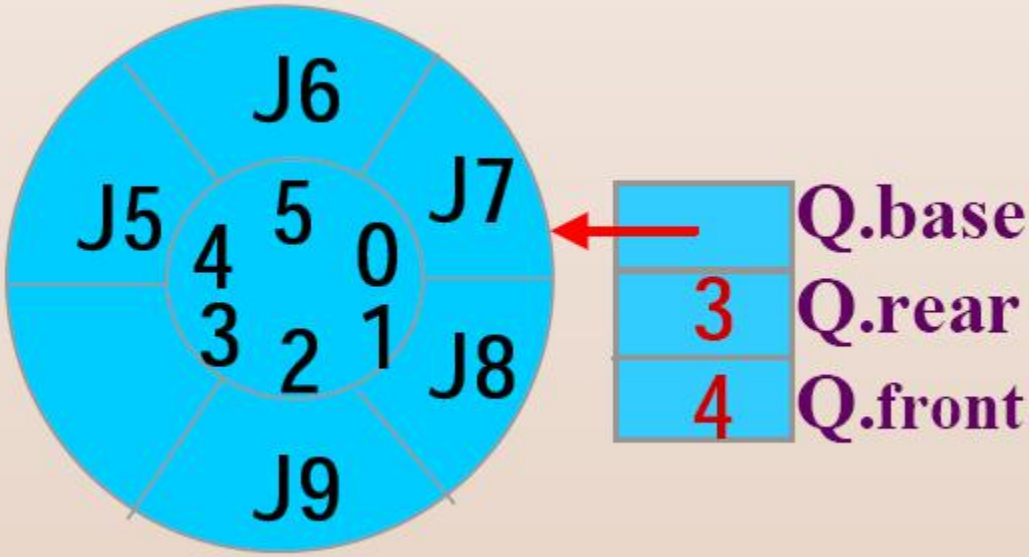
rear＝front，前者表示队满，后者表示队空

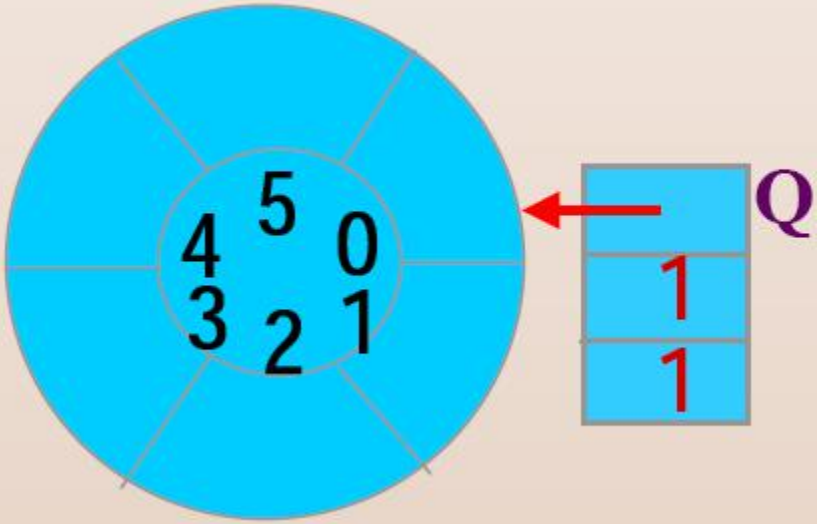
方法二

· 少用一个存储单元：约定以“队头指针在队尾指针的下一位置上”作为队满标志。

判满条件(Q.rear+1)%MAXQSIZE = =Q.front

判空条件Q.rear = =Q.front





循环队列——顺序映象

```
#define MAXQSIZE  100  //最大队列长度
  typedef struct {
    QElemType  *base;  // 动态分配存储空间
    int  front;     // 头指针，若队列不空，
                    //  指向队列头元素
    int  rear;      // 尾指针，若队列不空，指向
                    // 队列尾元素 的下一个位置
  } SqQueue;
```

```
Status InitQueue (SqQueue &Q) {
  // 构造一个空队列Q
  Q.base = (ElemType *) malloc
          (MAXQSIZE *sizeof (ElemType));
   if (!Q.base) exit (OVERFLOW);
                                  // 存储分配失败
   Q.front = Q.rear = 0;
    return OK;
}
```

```
Status EnQueue (SqQueue &Q, ElemType e)
{   // 插入元素e为Q的新的队尾元素
    if ((Q.rear+1) % MAXQSIZE == Q.front)
        return ERROR; //队列满
    Q.base[Q.rear] = e;
    Q.rear = (Q.rear+1) % MAXQSIZE;
    return OK;
}
```

```
Status DeQueue (SqQueue &Q, ElemType &e)
{  // 若队列不空，则删除Q的队头元素，
   // 用e返回其值，并返回OK; 否则返回ERROR
    if (Q.front == Q.rear)  return ERROR;
    e = Q.base[Q.front];
    Q.front = (Q.front+1) % MAXQSIZE;
    return OK;
}
```

3.5 队列的应用举例

- 如果将一个二项式（a+b）^{n}展开，它的系数能构成一个三角形，称它为**杨辉三角形**，也称它为帕斯卡三角形。利用队列输出杨辉三角形的前n行。

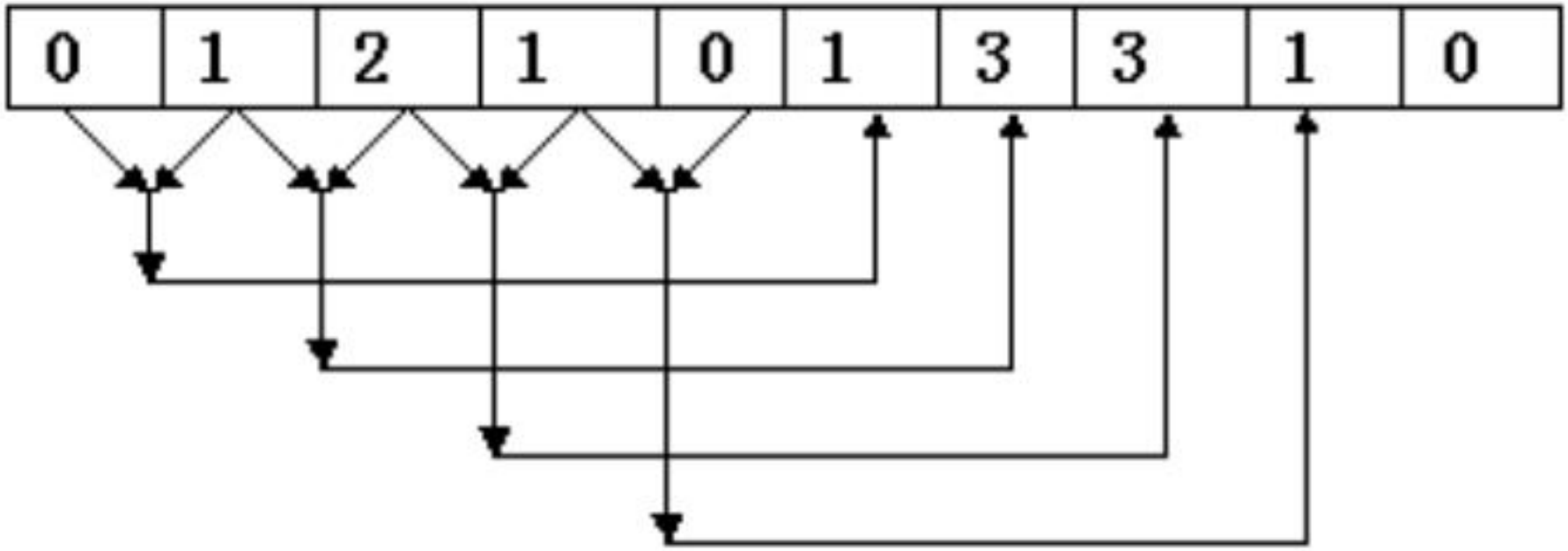
第 1 行				1		1		
第 2 行			1		2		1	
第 3 行		1		3		3		1
第 4 行	1		4		6		4		1

二项式系数值（杨辉三角）

第 1 行				1		1		
第 2 行			1		2		1	
第 3 行		1		3		3		1
第 4 行	1		4		6		4		1

设第 i-1行的值：(a[0]=0) a[1]..a[i] (a[i+1]=0)

则第 i 行的值：b[j] = a[j-1]+a[j], j=1,2,…,i+1

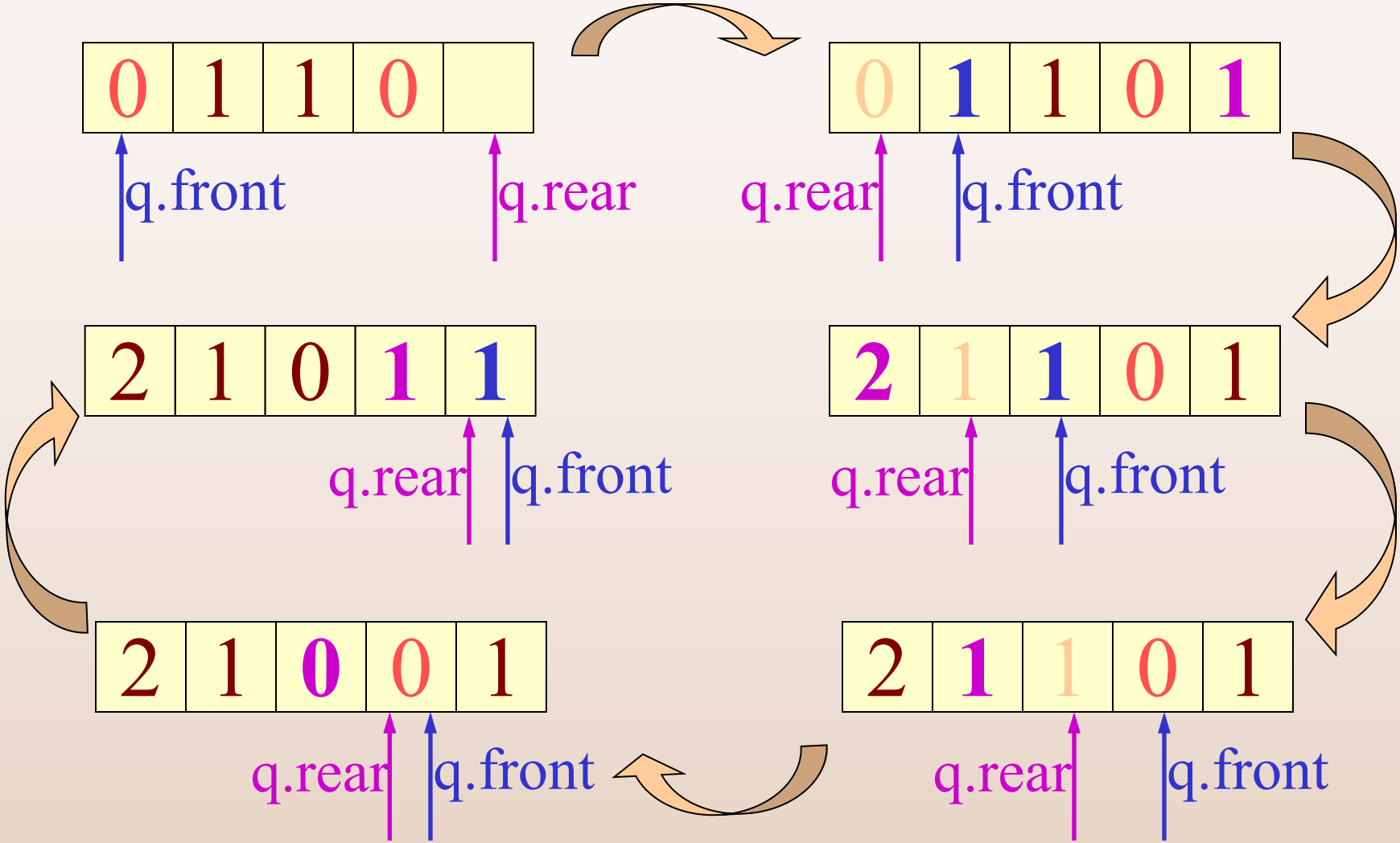

0
1
2
1
0
1
3
3
1
0

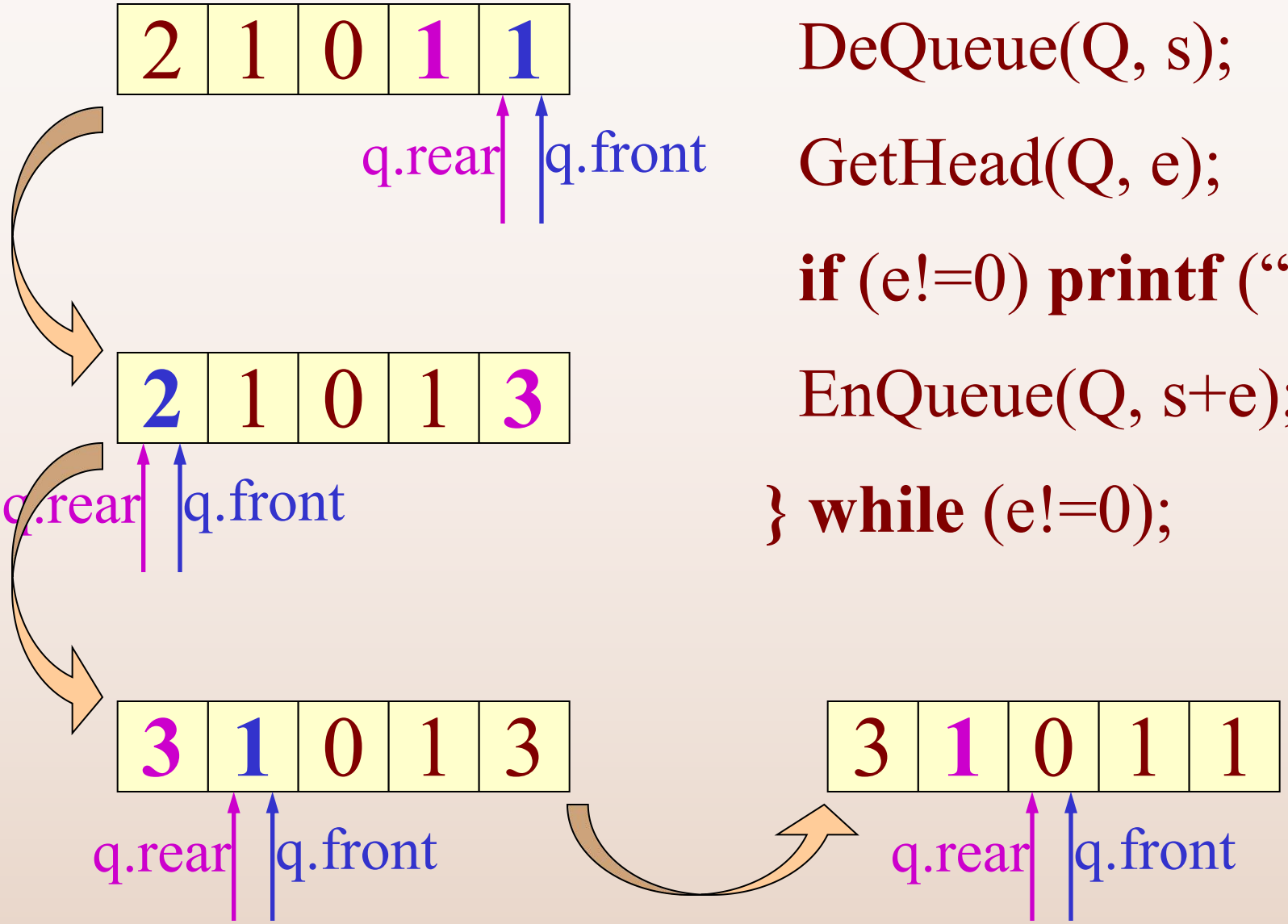
本章学习要点

1. 掌握栈和队列类型的特点，并能在相应的应用问题中正确选用它们。
2. 熟练掌握栈类型的两种实现方法，特别应注意栈满和栈空的条件以及它们的描述方法。
3. 熟练掌握循环队列和链队列的基本操作实现算法，特别注意队满和队空的描述方法。

利用循环队列计算二项式的过程：

假设只计算四行，则队列的最大容量为 5。






```
do {
    DeQueue(Q, s);
    GetHead(Q, e);
    if (e!=0) printf ("%d", e);
    EnQueue(Q, s+e);
} while (e!=0);
```